Capa de Transporte: TCP y UDP

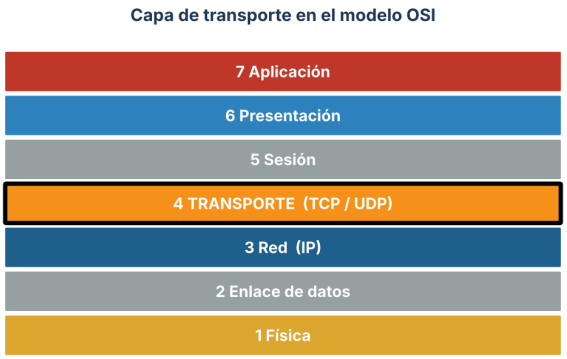
Resumen para el parcial · Telecomunicaciones

Idea central: IP sólo direcciona y enruta (sin garantías). La **capa de transporte** lleva los datos de una **aplicación a otra** usando **puertos**. **TCP** = confiable, lento, con estado. **UDP** = liviano, rápido, sin garantías.

1. Capa de transporte

Función

- Proporciona **comunicación extremo a extremo entre aplicaciones**: los datos de una app en un host llegan a la app correspondiente en el otro host.
- Usa **puertos** para identificar los procesos de origen y destino.
- Es la **capa 4 del modelo OSI**. Protocolos: **TCP** (Protocolo de Control de Transmisión) y **UDP** (Protocolo de Datagramas de Usuario).
- Ambos reciben datos de la capa de aplicación (TCP/IP) o sesión (OSI) y los pasan a la capa de internet (TCP/IP) o red (OSI).



Responsabilidades principales

- **Comunicación extremo a extremo** entre aplicaciones (con puertos).
- **Segmentación y reensamblado** de la información (aplicable a TCP): se divide en unidades del tamaño que soporta la red para evitar congestión en la red y en el receptor.
- **Entrega confiable, control de flujo y corrección de errores** (ej.: ordenamiento de segmentos) → lo hace **TCP**.

IP vs. capa de transporte

- **IP**: direcciona y enruta paquetes, **no garantiza entrega ni orden**.
- **TCP/UDP**: manejan **cómo se transfieren los datos** entre aplicaciones en distintos hosts.

2. Puertos y sockets

¿Qué es un puerto?

- Punto de conexión para el intercambio/transmisión de datos. Una app lo usa para **recibir ("escuchar")** y puede usar el mismo u otro para enviar.

Rango	Nombre	Detalle
0 – 1023	Well Known Ports	Conocidos / uso común. Los asigna **IANA**.

Rango	Nombre	Detalle
1024 – 49151	Registered Ports	Registrados / semi reservados. Empresas, organizaciones o individuos se registran con IANA.
49152 – 65535	Efímeros	Los usan los **programas cliente**.

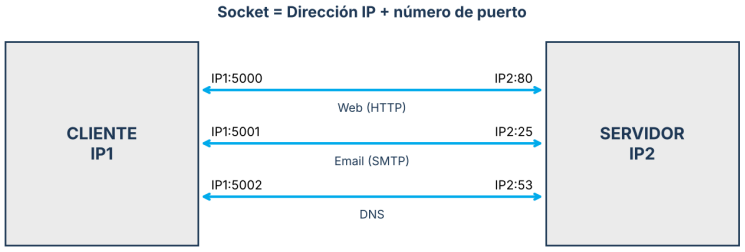
Puertos bien conocidos

Puerto	Protocolo	Aplicación
20 / 21	TCP	FTP (datos / control)
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	UDP y TCP	DNS
67, 68	UDP	DHCP
69	UDP	TFTP
80	TCP	HTTP (WWW)
110	TCP	POP3
161	UDP	SNMP
443	TCP	SSL / HTTPS
16384–32767	UDP	VoIP y video basado en RTP

Regla clave: un puerto sólo lo usa **una aplicación a la vez**. No pueden haber 2 apps escuchando en el **mismo puerto TCP de la misma IP**. Pero sí una en el **80 TCP** y otra en el **80 UDP** (son protocolos distintos).

¿Qué es un socket?

- Método de comunicación entre programas en una red. **Cada extremo de la conexión tiene un socket.**
- **Socket = Dirección IP + número de puerto.**



3. TCP (Transmission Control Protocol)

Características generales

- **Orientado a la conexión**: conecta dos aplicaciones de forma confiable. Definido en la **RFC 793 (septiembre de 1981)**; actualizada en 1122, 3168, 6093, 6528.
- Principales características: **confiable**, **entrega en el mismo orden** (ordena los segmentos si llegan desordenados) y **control de flujo**.

- Es **con estado (stateful)**: hace seguimiento de la sesión (desde el establecimiento hasta el cierre). Mantener ese estado consume recursos que un protocolo sin estado (UDP) no necesita.
- Apps típicas: navegadores web (HTTP/HTTPS), correo, transferencia de archivos (FTP), Telnet, bases de datos.

RFCs relacionadas (para reconocer)

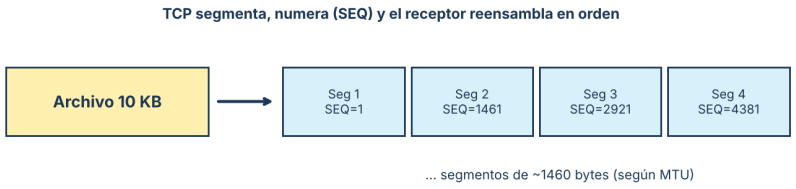
RFC	Tema	Qué trata
813	Window and Acknowledgment Strategy	Ventanas deslizantes de confirmación y sus problemas
879	TCP Maximum Segment Size	MSS: tamaño máx. de segmento y su relación con el datagrama IP
896	Congestion Control in IP/TCP	Problemas de congestión y cómo TCP los maneja
1122	Requirements for Internet Hosts	Cómo implementar TCP en hosts
1146	TCP Alternate Checksum	Mecanismo alternativo de checksum
1323	Extensions for High Performance	Enlaces de alta velocidad, nuevas opciones
2018	Selective Acknowledgment	Permite pedir retransmisión de segmentos específicos
5681	TCP Congestion Control	4 algoritmos de control de congestión
6298	Computing TCP's Retransmission Timer	Cálculo del **RTO**

Confiabilidad: PAR (Positive Acknowledgment with Retransmission)

- El emisor **envía datos y espera un ACK** (acuse de recibo).
- Si **no recibe el ACK** en un tiempo determinado, **retransmite** el segmento perdido.
- Sólo se retransmiten los datos **no confirmados**, no todo el mensaje.
- Operaciones básicas: **1)** seguimiento de segmentos enviados, **2)** acuse de recibo del receptor, **3)** retransmisión de lo no confirmado.

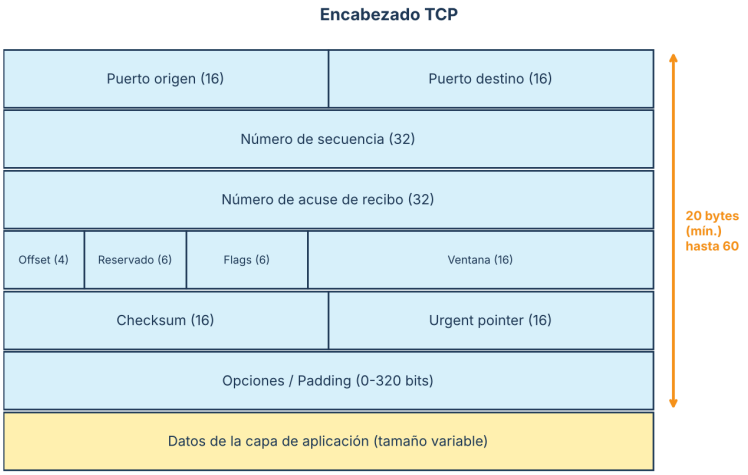
Segmentación y numeración

- TCP **divide los datos grandes en segmentos** que entran en el tamaño máximo de la red (**MTU**). Cada uno lleva un **número de secuencia** → el receptor los **reensambla en orden**.
- Ejemplo: archivo de 10 KB → varios segmentos de ~1460 bytes, secuenciados.



Encabezado TCP

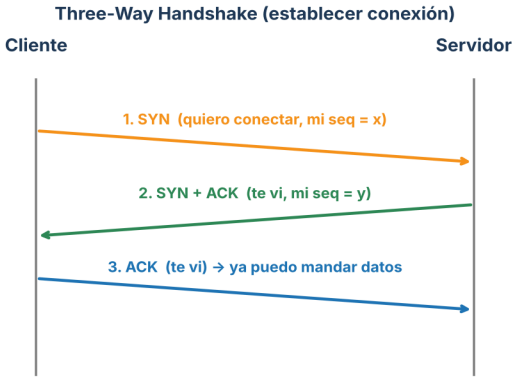
- Parte fija de **20 bytes** + opciones de hasta **40 bytes** (total máx. 60). Mucho más grande que el de UDP (8 bytes) → **más sobrecarga**.



Campo	Bits	Descripción
Source Port	16	Puerto del proceso que originó el mensaje
Destination Port	16	Puerto del proceso destinatario
Sequence number	32	Si **SYN=1**: es el número de secuencia inicial (ISN); el 1.er byte de datos será ISN+1. Si **SYN=0**: es el número acumulado del 1.er byte de datos del segmento.
Acknowledgment number	32	Si ACK está activo: es el **próximo número de secuencia que el receptor espera**. Confirma todos los bytes anteriores.
Data offset	4	Tamaño de la cabecera en palabras de 32 bits (mín. 5 = 20 bytes; máx. 15 = 60 bytes). También es el desplazamiento hasta los datos.
Reserved	6	Uso futuro (se ponen en 0)
Flags (6)	6	**URG** datos prioritarios · **ACK** campo ack válido · **PSH** enviar buffer a la aplicación · **RST** aborta la conexión por error · **SYN** inicia conexión / sincroniza secuencias · **FIN** no hay más datos del emisor
Window size	16	Tamaño de ventana de recepción: bytes que el remitente está dispuesto a recibir
Checksum	16	Control de errores de header y datos
Urgent pointer	16	Con URG activo, indica cuántos datos (desde el 1.er byte) son urgentes
Options	0–320	En bloques de 32 bits (ver RFC 793)

Establecimiento de conexión: Three-Way Handshake

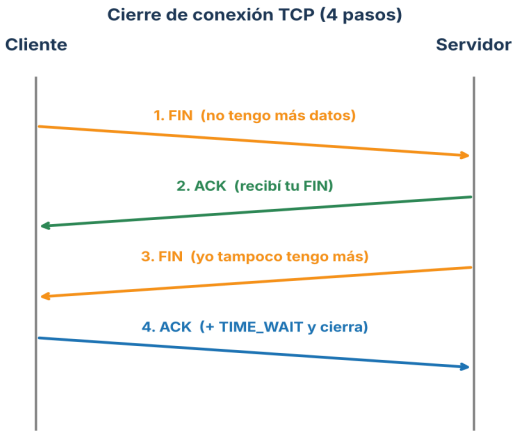
- TCP **negocia y establece una sesión permanente** entre origen y destino **antes** de enviar datos. Así los dispositivos negocian cuánto tráfico se puede reenviar a la vez.
- **Paso 1 – SYN:** el cliente pide la conexión (segmento sin datos, sólo cabecera).
- **Paso 2 – SYN + ACK:** el servidor confirma el SYN y a la vez indica que él también quiere conectar.
- **Paso 3 – ACK:** el cliente confirma el SYN del servidor → **conexión establecida** (ambos pasan a ESTABLISHED).
- Después del handshake, se envían los datos en segmentos con encabezado que garantizan entrega confiable, ordenada y sin errores.



Estados (según el diagrama del material): Cliente: CLOSED → SYN-SENT → ESTABLISHED. Servidor: CLOSED → LISTEN → SYN-RECEIVED → ESTABLISHED.

Cierre de conexión (4 pasos)

- **1.** Un extremo envía **FIN** (sin datos, sólo cabecera): pide cerrar.
- **2.** El otro extremo responde **ACK**. Quien envió el FIN ya **no puede enviar más datos nuevos** → es el cierre de **un solo sentido**.
- **3.** El otro extremo envía **su propio FIN**.
- **4.** Se confirma con **ACK**. Quien lo envió espera un tiempo (**TIME_WAIT**) por si el ACK se pierde y hay que reenviarlo. Luego, conexión **cerrada**.

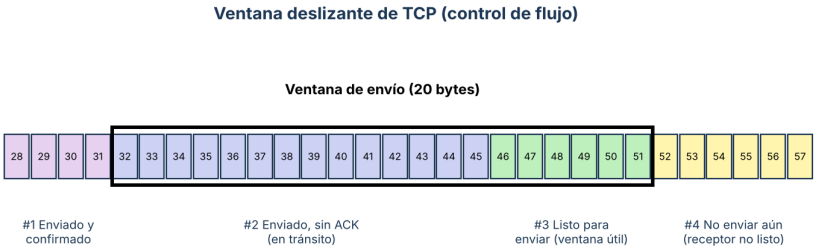


Entrega en el mismo orden: ISN y RTO

- Las redes pueden usar **múltiples rutas** → los datos pueden llegar **fuera de orden**. TCP **numera y secuencia** los segmentos para rearmarlos antes de entregarlos a la app.
- **Reencarnación:** reutilizar una conexión (mismo par IP/puerto). Para no confundir con segmentos viejos que aún circulan, TCP usa un **ISN (Initial Sequence Number) distinto en cada conexión**.
- **ISN:** número de **32 bits**, generado por un **reloj interno que se incrementa cada 4 microsegundos** → único y variable por conexión.
- Gracias al ISN, el receptor **identifica y descarta segmentos duplicados** de una conexión anterior.
- **RTO (Retransmission Timeout):** cuánto espera el emisor antes de **retransmitir un segmento no confirmado** (sin ACK). Fórmula en la **RFC 6298**.
  - RTO muy chico → retransmisiones innecesarias. RTO muy grande → reacción lenta ante pérdidas.

Control de flujo (ventana deslizante)

- TCP usa una variación de **ventanas deslizantes**. El acuse de recibo está **orientado al flujo de bytes** (cuenta bytes, no segmentos).
- El ACK es **acumulativo**: acusar el número de secuencia **X** significa que **todos los octetos hasta (sin incluir) X** fueron recibidos.
- El campo **Window size (16 bits)** indica cuántos bytes puede aceptar y procesar el destino a la vez. El **tamaño inicial se acuerda en el three-way handshake**.
- El origen **limita lo que envía según la ventana** y sólo sigue mandando cuando recibe ACK. Mientras espera el ACK, no manda otro segmento.
- Si la red o el receptor se congestionan, la demora aumenta y **baja la tasa de transmisión efectiva** → menos conflicto de recursos. Se evita pérdida de segmentos y retransmisiones.
- TCP busca **subir la velocidad hasta el máximo** que soportan la red y el destino, minimizando pérdidas y retransmisiones.
- Numeración de octetos: el primer octeto tras la cabecera es el de menor número. El espacio de secuencia es finito: de **0 a $2^{32} - 1$**.



Del ejemplo: 14 bytes ya enviados sin ACK + 6 bytes de **ventana útil** = ventana de envío de 20 bytes. Al llegar ACKs, la ventana se **desliza** a la derecha.

Costo de la confiabilidad

- **Sobrecarga** por: mensajes de control, seguimiento de estado y retransmisiones → puede provocar **demoras**.
- Pero es necesario para apps que requieren **entrega completa y ordenada**.
- Es más eficiente cuando se envían **grandes volúmenes de datos**; para datos pequeños o esporádicos la sobrecarga es innecesaria (conviene UDP).

4. UDP (User Datagram Protocol)

Qué es

- **RFC 768 (1980).** Protocolo de transporte **liviano**, de **entrega de máximo esfuerzo (best-effort)**, sin la confiabilidad ni el control de flujo de TCP.
- “Poco confiable” porque **no hay acuse de recibo** que indique que los datos llegaron.
- Analogía: **carta simple sin registrar**. El emisor no sabe si el receptor está disponible, ni el correo hace seguimiento ni avisa si no llega.
- TCP y UDP son ambos válidos: el **desarrollador elige** según los requisitos de la aplicación (a veces se usan ambos).
- Es un protocolo **sin estado (stateless)**: ni cliente ni servidor llevan seguimiento de la sesión. Cada datagrama se maneja de forma independiente.

¿Qué hace UDP? (sólo 3 tareas)

- **1. Recibir datos de la capa superior:** una app le envía datos.
- **2. Encapsular:** el mensaje se mete en el campo de datos del mensaje UDP y se completa la cabecera (puerto origen y destino).
- **3. Pasar a IP:** entrega el mensaje UDP a la capa 3 para transmitirlo.

¿Qué NO hace UDP?

- No establece conexiones antes de enviar: **simplemente empaqueta y envía**.
- No da garantías de que los mensajes lleguen ni **reconocimientos (ACK)**.
- **No detecta pérdidas ni retransmite.**
- **No asegura el orden** de recepción.
- **No gestiona flujo ni congestión.**

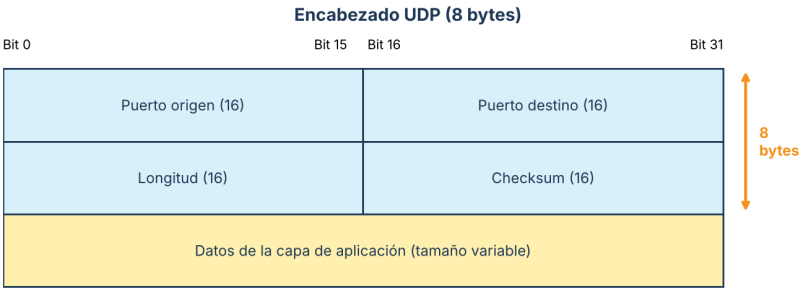
Características (las 4 del esquema)

Característica	Detalle
Sin conexión	No establece sesión entre hosts antes de enviar/recibir datos.
Entrega no confiable	Sin servicios para asegurar la entrega; no hay procesos para retransmitir datos perdidos o dañados.
Reconstrucción no ordenada	Los datos pueden llegar en otro orden; UDP no los reordena: **los entrega a la app en el orden en que llegan**.
Sin control de flujo	No limita cuánto envía el origen. Si el destino se sobrecarga, **descarta datos** hasta tener recursos, y no hay retransmisión automática.

La baja sobrecarga lo hace ideal para apps que **toleran cierta pérdida de datos**. Su PDU se llama **datagrama** (aunque “segmento” y “datagrama” a veces se usan indistintamente para la PDU de transporte).

Encabezado UDP

- Sólo **8 bytes** (vs. 20–60 de TCP). 4 campos de 16 bits.



Campo	Bits	Descripción
Source Port	16	Puerto del proceso que originó el mensaje. Suele ser puerto dinámico (cliente → servidor) o bien conocido/registrado (servidor → cliente).
Destination Port	16	Puerto del proceso destinatario. Suele ser bien conocido/registrado (petición a servidor) o dinámico (respuesta a cliente).
Length	16	Longitud de **todo** el datagrama (cabecera + datos).
Checksum	16	**Opcional**. Suma de comprobación de todo el datagrama. Si no se usa, se pone en todos unos.
Data	variable	Mensaje de la capa superior.

Fragmentación y segmentación

- **TCP:** cada segmento tiene **número de secuencia** → el destino reensambla en orden y la app recibe los datos como los planeó el emisor.

- **UDP: no tiene número de secuencia** ni gestiona segmentación internamente.
- Si los datos superan el tamaño máximo del datagrama, **la capa IP puede fragmentarlos**. UDP **no reensambla ni garantiza el orden** de los fragmentos: esa responsabilidad **recae en la aplicación**.
- Si se necesita confiabilidad sobre UDP, **la debe administrar la aplicación**.

Cómo se comunica UDP

- Está **basado en transacciones**: cuando la app tiene datos, simplemente los envía. No hay sesiones previas.
- Muchas apps UDP mandan **pocos datos que entran en un solo segmento**.
- A los servidores UDP se les asignan **puertos bien conocidos o registrados**, igual que a los de TCP. Al recibir un datagrama, UDP lo entrega a la app según el **puerto destino**.
- El cliente elige puertos origen y destino y **ese mismo par se usa en todos los datagramas de la transacción**.
- En la **respuesta** del servidor al cliente, los puertos origen y destino se **invierten**.
- Que UDP no sea confiable **no significa que sea un protocolo "inferior"**: esas funciones simplemente se implementan en la app si hacen falta.

¿Por qué algunas aplicaciones usan UDP?

- **1.** El **rendimiento** es más importante que la fiabilidad.
- **2.** Intercambios de datos **cortos y relevantes**.
- **3. Tráfico multicast.**

Si juegos online o VoIP usaran TCP, tendrían **largas demoras** por la detección de pérdidas y retransmisión: la demora perjudica más que una pequeña pérdida. Apps como DNS directamente **reintentan** la solicitud si no hay respuesta.

Los 3 tipos de aplicaciones más adecuadas para UDP

- Apps que **toleran cierta pérdida** pero necesitan **retrasos cortos o nulos** (VoIP, streaming, IPTV, juegos).
- Apps con **transacciones de solicitud/respuesta simples** (DHCP, DNS, SNMP, TFTP).
- **Comunicaciones unidireccionales** donde no se requiere confiabilidad o la app la puede administrar.

Protocolos que usan UDP: DNS (también puede usar TCP), SNMP, DHCP, RIP, TFTP, VoIP, IPTV, juegos en línea.

> **TFTP** es el ejemplo de app que se ocupa de la confiabilidad por sí misma: tiene **sus propios** mecanismos de control de flujo, detección y recuperación de errores y acuses de recibo, por eso no depende de TCP.

5. Cuadro comparativo TCP vs UDP

Característica	TCP	UDP
Conexión	**Orientado a la conexión** (three-way handshake previo)	**Sin conexión**: envía de inmediato
Estado	**Stateful**: mantiene estado en ambos extremos (secuencias, ventana, etc.)	**Stateless**: cada datagrama es independiente
Modelo de esfuerzo	Hace todo lo posible por entregar correcta, ordenada y completa	**Best-effort**: sin garantías; si se pierde, no se recupera
Confiabilidad	**Alta**: ACKs, retransmisión, control de secuencia	**Baja / no confiable**: sin ACK, sin retransmisión, no verifica si el destino está listo

Característica	TCP	UDP
Control de flujo y congestión	**Sí**: ventanas deslizantes	**No**: envía a la máxima velocidad
Segmentación y reensamblado	**Sí**: segmentos numerados, reensamblado garantizado	**No**: lo hace la app (capa 7) o IP (fragmentación)
Orden de entrega	Ordenada	No ordenada
Overhead (cabecera)	**Alto**: 20–60 bytes	**Bajo**: 8 bytes (puertos, longitud, checksum)
Velocidad	Más lenta	**Más rápida**; ideal para tiempo real
Mensajes de control	Muchos (handshake, ACK, etc.)	Muy pocos
Aplicaciones típicas	Web (HTTP/HTTPS), email (SMTP, IMAP, POP), FTP, SSH, Telnet, bases de datos	Streaming, VoIP, videollamadas, juegos online, DNS, SNMP, DHCP, TFTP
PDU	Segmento	Datagrama

6. Puntos para repasar antes del parcial

- **TCP:** RFC 793 (1981). **UDP:** RFC 768 (1980).
- Cabecera TCP: **20 bytes** (hasta 60) · Cabecera UDP: **8 bytes**.
- **Three-way handshake:** SYN → SYN+ACK → ACK. **Cierre:** FIN → ACK → FIN → ACK (+ TIME_WAIT).
- Flags TCP: **URG, ACK, PSH, RST, SYN, FIN**.
- **ISN:** 32 bits, reloj interno que sube cada 4 µs; evita confundir segmentos de “reencarnaciones”.
- **RTO** = tiempo de espera antes de retransmitir (RFC 6298). **PAR** = Positive Acknowledgment with Retransmission.
- ACK **acumulativo**: ACK = X confirma todo hasta X−1. Espacio de secuencia: 0 a $2^{32}-1$.
- Puertos: **0–1023** conocidos (IANA) · **1024–49151** registrados · **49152–65535** efímeros (clientes).
- **Socket = IP + puerto.** Un mismo número de puerto puede usarse en TCP y UDP a la vez (80/TCP y 80/UDP), pero no 2 apps en el mismo puerto TCP de la misma IP.
- **DNS:** UDP (y TCP también). **DHCP:** UDP 67/68. **TFTP:** UDP 69. **SNMP:** UDP 161. **HTTP:** TCP 80. **HTTPS/SSL:** TCP 443. **SMTP:** TCP 25. **SSH:** TCP 22.
- Si se necesita confiabilidad sobre UDP, **la implementa la aplicación**.